Malware Analysis for Digital Forensics

Dr Olga Angelopoulou

ROYAL HOLLOWAY UNIVERSITY OF LONDON

Overview

- The importance of malware analysis in a digital forensics investigation
- Selected concealment techniques
- Issues in detection and mitigation of malware

Introduction to Malware

ROYAL HOLLOWAY UNIVERSITY OF LONDON

Malicious Software Taxonomy

Malicious software (or **Malware**) is a summary term for a broad range of software that is executed without the consent (or approval) of legitimate users.

- software that violates a system's **security policy**.

A precise taxonomy is not feasible as the categories tend to blur, but the following broad taxa can be identified:

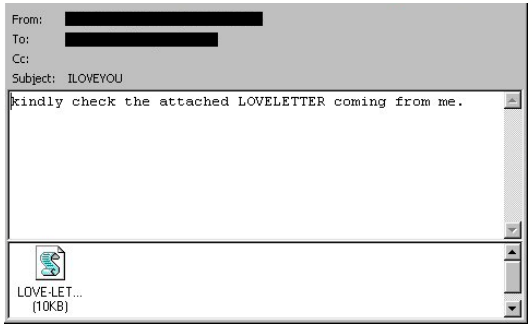
- **Viruses:** Code that inserts itself into one or more (executable) files and typically performs a malicious function
- **Worms:** Code that is capable of replicating itself autonomously on remote systems
- **Trojan Horses:** Code with an overt (cover) function and a covert, typically malicious secondary function
- **Ransomware:** Prevents users from accessing their data or systems and demands a payment, typically in cryptocurrency, in exchange for restoring access.

The Big Picture

- Hundreds of thousands of new malware samples identified daily
- Ransomware remains one of the most financially damaging threats
- Malware is now:
  - Largely profit-driven
  - Increasingly human-operated
  - Often part of multi-stage attacks
  - Small and medium organisations are also frequent targets

Malware Statistics

- Around 1% of all businesses experienced a ransomware crime in 2025, equating to approximately 19,000 affected organisations - up significantly from 2024. - Link
- In 2024 there were 5,414 publicly disclosed ransomware attacks against organisations worldwide, an 11% increase compared to 2023. – Link
- 94% of malware is delivered via email, while 90% of cyber threats are phishing or social engineering schemes.
- In the second quarter of 2025, approximately 143,000 malicious mobile packages were detected, with Trojan-Bankers, RiskTools, and AdWare being the most common types. 92% of mobile malware in 2023 targeted Android devices.
- The education sector, followed by government and military, experienced the highest volume of weekly malware attacks in 2022.
- Phishing attacks increased by 1,265% largely driven by generative AI.



Ransomware attacks surge 69% across global education sector

Ransom demands averaged $608,000 for international education entities in the first quarter of 2025, a Comparitech analysis found.

Published April 14, 2025

Link

Ransomware trends, statistics and facts in 2026

Supply chain attacks, triple extortion, GenAI and RaaS are some of the ransomware trends that will continue to disrupt businesses in 2026. Is your industry a top target?

By Sean Michael Kerner Published: 06 Jan 2026

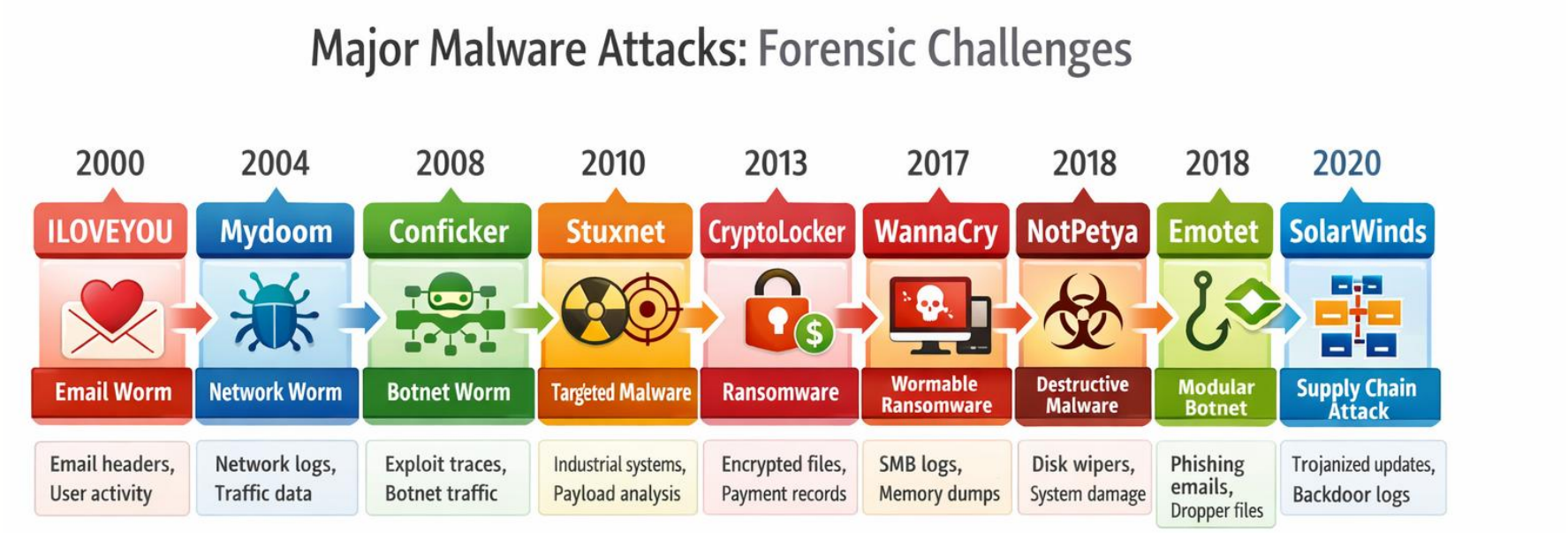
Link

Linux Malware and Vulnerability Key Statistics

- Linux kernel CVEs increased 1,117% in 2024 to 3,529 vulnerabilities, with 5,530 recorded in 2025 year-to-date as of early January.
- Webshells account for 49.6% of all Linux malware exploits, targeting web servers as primary attack vectors.
- Linux ransomware attacks increased 62% between 2022 and 2023, with ESXi servers facing average ransom demands of $5 million.
- Brute-force attacks represent 89% of all endpoint behaviors on Linux systems, focusing on SSH endpoints in public-facing infrastructure.
- Linux accounts for only 1.3% of global malware detections in 2025 despite powering 90% of public cloud workloads.

Link

Major Malware Attacks



Malware evolved from disruption to industrialised cybercrime and strategic attack, fundamentally changing digital forensic practice.

Generated with the support of ChatGPT

Malware Analysis in Digital Forensics

ROYAL HOLLOWAY UNIVERSITY OF LONDON

Malware analysis as part of a digital forensic investigation

ROYAL HOLLOWAY UNIVERSITY OF LONDON

- Focus on understanding what the malware did, not just how it works
  - behaviour,
  - impact,
  - persistence,
  - how malware fits into a wider timeline of events
- Supports attribution, impact assessment, and incident reconstruction

Malware Analysis vs Traditional Forensics

Traditional forensics

Retrospective

Evidence-focused

Malware analysis

Behaviour-focused

Often time-sensitive

Both require:

- Documentation
- Repeatability
- Caution

Malware lifecycle

Initial Access	Execution	Persistence	Command & Control	Actions on Objectives	Cleanup / Anti-Forensics
•Phishing •Exploits •Malicious downloads	•Payload runs on the system	•Survives reboot or user action	•External communication	•Data theft •Encryption •Lateral movement	•Log removal •Obfuscation •Self-deletion

Malware Lifecycle to Forensic Artefacts

Malware lifecycle stage	Typical forensic artefacts
Initial Access	Email artefacts, browser history, exploit logs, web server logs
Execution	New processes, executable files, memory artefacts
Persistence	Registry Run keys, scheduled tasks, services, startup folders
Command & Control	DNS queries, network logs, proxy/firewall logs
Actions on Objectives	File access, encryption activity, credential artefacts
Cleanup / Anti-Forensics	Deleted files, cleared logs, timestamp anomalies

Malware Lifecycle and MITRE ATT&CK

MITRE ATT&CK framework provides a structured framework for describing adversary behavior.

Malware lifecycle stage	MITRE ATT&CK tactic
Initial Access	Initial Access
Execution	Execution
Persistence	Persistence
Command & Control	Command and Control
Actions on Objectives	Exfiltration / Impact
Cleanup / Anti-Forensics	Defense Evasion

MITRE ATT&CK in Forensics

ROYAL HOLLOWAY UNIVERSITY OF LONDON

- Standardises descriptions of attacker behavior
- Supports incident reconstruction
- Improves communication between DFIR teams
- Helps avoid vague or speculative reporting

Forensic Analysis of Malware (1)

ROYAL HOLLOWAY UNIVERSITY OF LONDON

The focus of this block is on **autonomous propagation** and **obfuscation** techniques

- Viruses and Trojan Horses require some level of human interaction and are hence more difficult to describe regarding their propagation behavior but share many of the obfuscation techniques used
- Trapdoors and Logic Bombs can be considered **payloads,** while analyzing their effects and mechanisms is a vital part of a forensic investigation, these tend to be more specialized and therefore difficult to summarize
- Some uncertainty exists as **adware** and **spyware** is not always classified consistently

Forensic Analysis of Malware (2)

ROYAL HOLLOWAY UNIVERSITY OF LONDON

Forensic analysis of malware can occur in very different contexts:

- Identifying novel malware is a specialized task normally performed by anti-virus researchers and vendors
- For **targeted** (customized) malware, this may be needed in case of specific attacks, and as a result of obfuscation mechanism

More specifically, forensic analysis usually is called upon to establish whether a certain activity has taken place (or could have taken place):

- Establishing the mere presence of malware or even whether the malware **could** have performed a certain action is only a necessary, but not sufficient condition

Forensic artefacts recovery

- As malware evolved, forensic certainty decreased and incident response became inseparable from investigation.

  - Early malware → **clear artefacts, low sophistication**
  - Mid-era targeted malware → **complex artefacts, attribution challenges**
  - Modern attacks → **distributed evidence, heavy DFIR reliance**

Malware Concealment and Detection Techniques

Impossibility of Detection

Cohen, 1984: Regardless of techniques used for detection mechanisms, it is **impossible** to construct a perfect malware or virus detector.

The proof sketch for this relies on a **diagonalization** argument:

- Assume P to be a perfect detector, M an arbitrary malware
- M may call or otherwise interact with P
- If $P(M) = \top, M$ will halt
- If $P(M) = \bot, M$ will spread
- Proof by contradiction: If P is a perfect detector, this would solve the **Halting Problem**

Binary Modification Strategies

ROYAL HOLLOWAY UNIVERSITY OF LONDON

Binary modification strategies are not limited to viruses but may be employed by any malware requiring persistent executable files or dynamic modification of processes.

Substitution	•Malware will simply overwrite a targeted executable or memory block; altered function may be trivial to identify
Pre-pending	•Malware payload is pre-pended to targeted executable and executed first or instead of target; must be accompanied by obfuscation of file sizes, signatures, and relative addresses
Appending	•Requires only a well-known entry point or modified start, may reside in unclaimed areas
Cavity	•If entry points can be specified dynamically, injecting code in a targeted executable at an arbitrary location can make detection substantially more difficult. Dynamic relocation allows insertion of further code, which can also be protected by obfuscation techniques
Multi-Cavity	•To further obfuscate code, malware payloads can be split into multiple segments and inserted into different areas of an executable or other target location.

Packers

A number of obfuscation techniques can be used to masquerade executable code, one common approach is the use of (encrypting) **packers**

- This also has the benefit of minimizing the size of the malware payload

Packers consist of a de-compression engine which is executed first and may compress both the malware payload and potentially also the target executable

- Typical features include compression and encryption
- Packers themselves often employ polymorphism (randomization of semantically equivalent code)
- Anti-debugging and anti-forensics techniques are used to obfuscate code structure and observability, as is VM presence detection

Root Kits

- Malware must also apply concealment mechanisms after a successful compromise to prevent detection in the compromise stage but particularly after a host has been subverted. This is accomplished by root kits:
- A root kit is a set of software tools and mechanisms inserted by an adversary into a target system
  - conceal the presence of modifications performed by the adversary
  - permit the adversary to take control over the compromised system with elevated privileges over an undetected channel.

Types of Rootkits

1. Userland (Binary) Rootkits
   i. Operate in user space by modifying executables or injecting code
   ii. Forensic implications: Manipulate output of system tools, may evade in-system analysis
   iii. Techniques
      - Off-line disk imaging
      - Hash/integrity verification
      - Raw memory inspection
      - Cross-tool validation

2. Kernel Rootkits
   i. Modify kernel structures or drivers (e.g., DKOM, hooking)
   ii. Forensic implications: Harder to detect using standard OS tools, can hide processes, drivers, and network connections at low level
   iii. Detection often requires:
      - Memory forensics
      - Kernel structure analysis
      - Cross-view comparisons
      - Integrity checking of drivers/modules

3. Persistent Rootkits
   i. Survive reboot by reinitialising from non-volatile storage
   ii. Common persistence vectors: Registry run keys, Startup files, Boot loader or driver modification, Firmware/boot record changes
   iii. Forensic implications: Evidence may exist both on disk and in memory
   iv. Requires:
      - Startup artefact analysis
      - Boot sector examination
      - Firmware integrity checks
      - Timeline reconstruction

Analysis Techniques

ROYAL HOLLOWAY UNIVERSITY OF LONDON

Goals of Malware Analysis

- Determine how the system was compromised
- Identify what actions the malware performed
- Assess data loss or system damage
- Support timeline reconstruction
- Inform remediation and reporting

Malware Analysis Techniques

Assuming code (and possibly static data) is acquired, e.g., from a memory or storage image

- Code or fragments can be matched against known signatures: Sometimes code is re-used, even if the file or memory image itself is unknown
- Most malware will have (in addition to concealment such as used by rootkits) some form of obfuscation built in, beginning with **packers** and **encryption** engines, limiting simple signature analysis and carving techniques

Instead, **static** or **dynamic** analysis methods are required for reverse engineering malware and understanding its actions

Static Analysis Techniques

ROYAL HOLLOWAY UNIVERSITY OF LONDON

Analyzing the executable code beginning with the executable format and unpacking (or code fragment) requires understanding of any dependencies on libraries and services

- Dependencies may also exist on context such as configuration data (e.g., registry keys in Windows)

Ultimately some form of disassembly (since debug symbols are unlikely to be included!) is needed

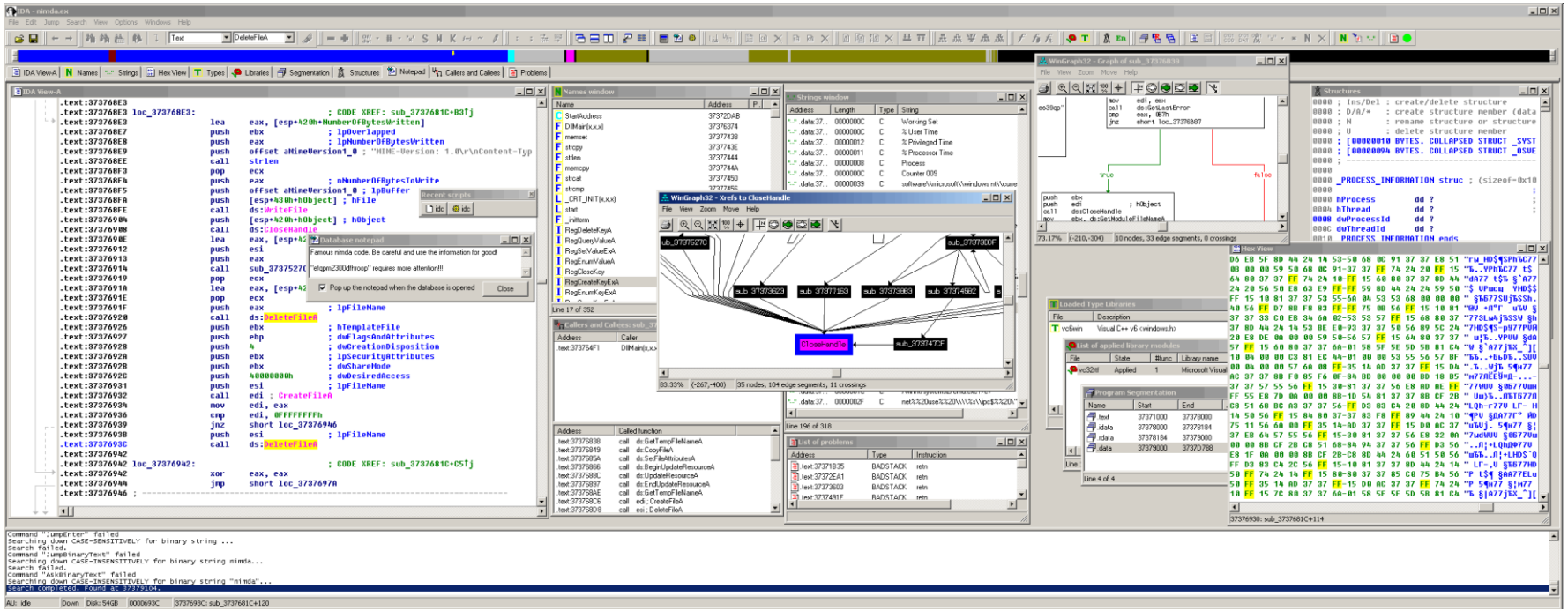
- Techniques such as recursive analysis allow the identification of **control flows** and interactions
- Most malware will employ obfuscation and counter-forensic techniques that fool some or all tools – requiring some manual intervention

Given the complexity of malware, tool support is still important, however

- Popular tools include IDA Pro or Hopper, or open-source alternatives such as Radare2, supporting different platforms and processor architecture

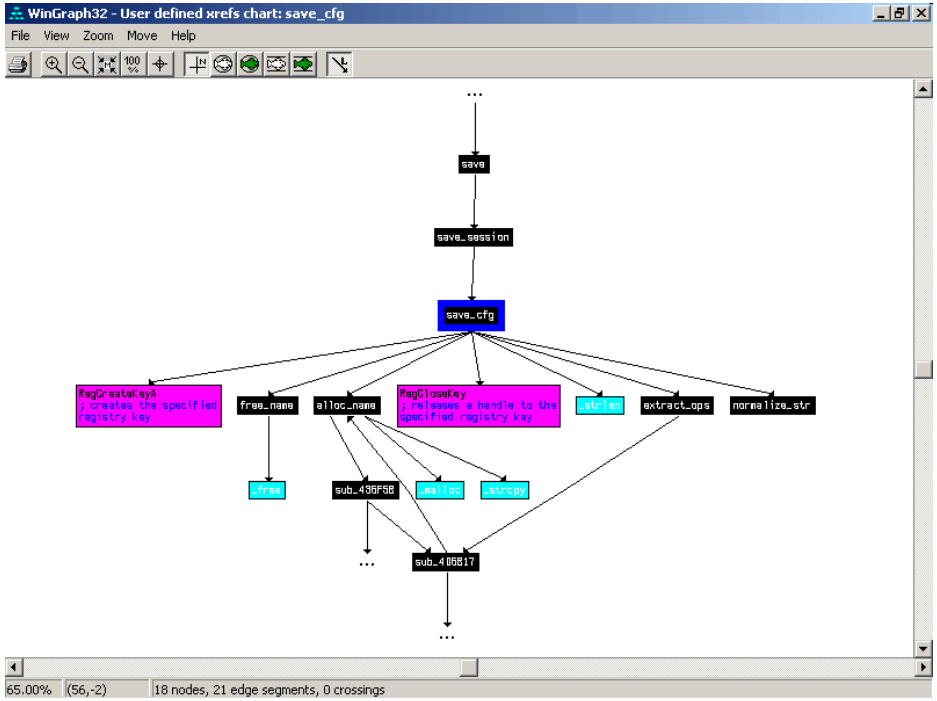
Static Analysis Tools (1/4)

IDA Pro (HexWorks) work flow:



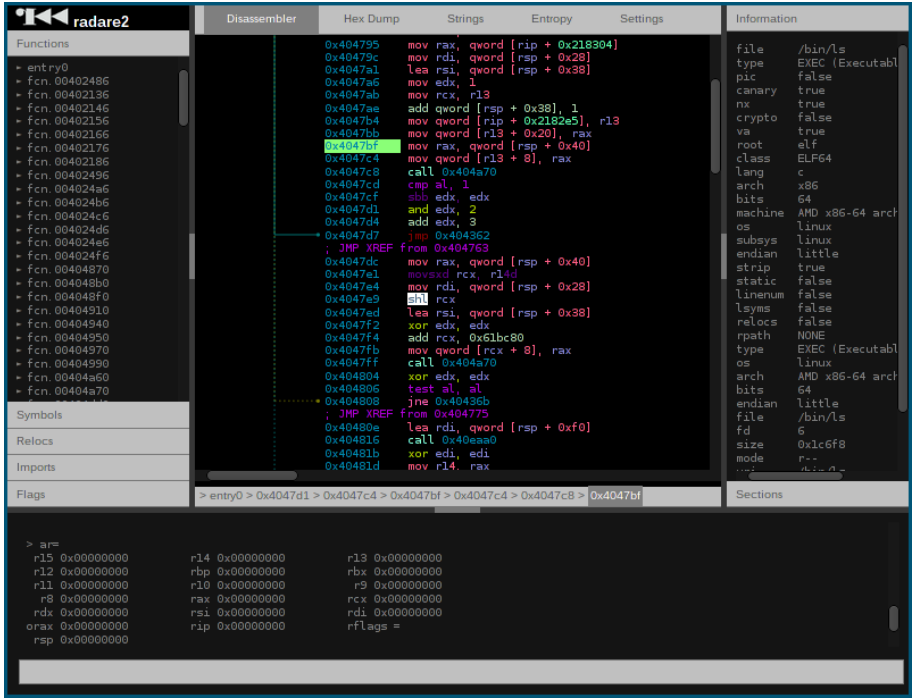
Static Analysis Tools (2/4)

Control flow reconstruction in IDA Pro:



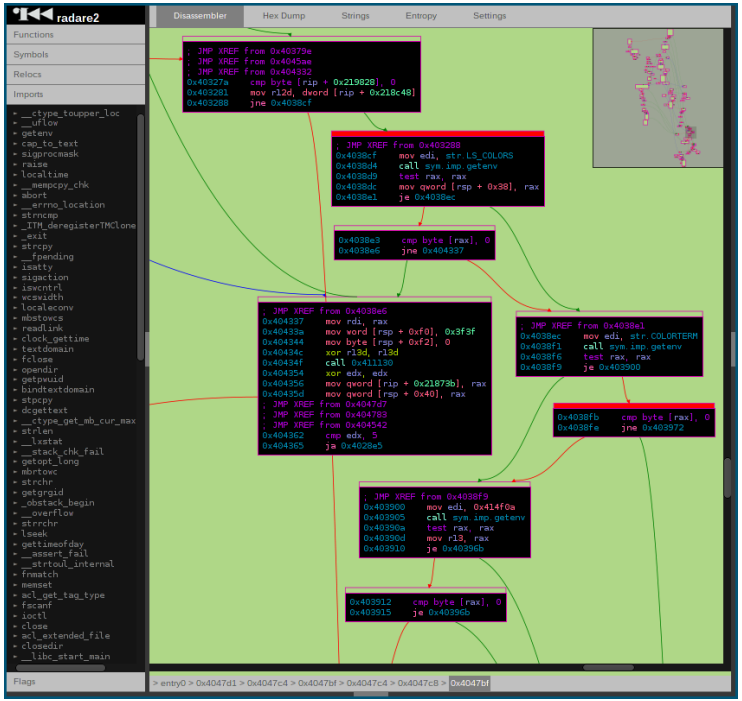
Static Analysis Tools (3/4)

Radare user interface:



Static Analysis Tools (4/4)

Radare control flow reconstruction with graphs and code fragments:



Dynamic Analysis

ROYAL HOLLOWAY UNIVERSITY OF LONDON

Static analysis gives good coverage of code, but may miss elements of the code and is rather resource intensive

- This can make **dynamic analysis**, i.e., observing code during run-time necessary
- This applies to all kinds of malware from executable binaries to script code executed as part of a drive-by download
- Malware will employ cloaking and obfuscation, monitoring for observation mechanisms
- Dynamic analysis environment must appear to be as close to the real, vulnerable system as possible and may need to elicit different code paths
- Malware will seek to detect modified environment with instrumentation for analysis…

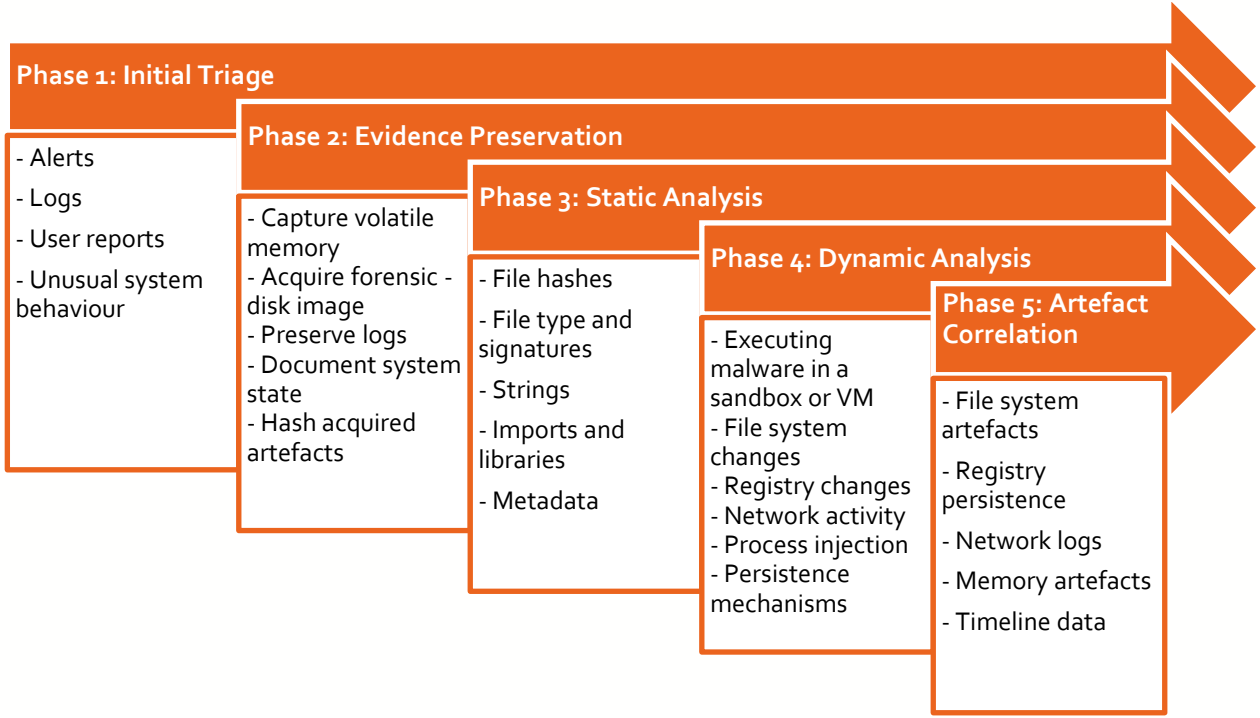
Discussion Point

Should malware analysts prioritise stopping an attack or preserving evidence?

Can both always be achieved?

Forensic Guidance

- Malware investigation should:
  - Be evidence-led
  - Follow forensic principles!
  - Maintain integrity and reproducibility



Artefacts of Malware

ROYAL HOLLOWAY UNIVERSITY OF LONDON

Host-Based Artefacts of Malware

- New or modified files
- Registry persistence mechanisms
- Scheduled tasks or services
- Unusual processes

Network-Based Artefacts

- Command-and-control traffic
- DNS anomalies
- Repeated outbound connections
- Encrypted or obfuscated traffic

Use with caution!

1. Persistence Mechanisms
   i. Registry Run keys
   ii. Startup folders
   iii. Scheduled tasks
   iv. Services and drivers
2. Anti-Forensic Techniques
   i. Obfuscation and packing
   ii. Encryption
   iii. Timestamp manipulation
   iv. Disabling logging or security tools

Legal and Ethical Considerations

- Lawful authority to analyse malware
- Handling live systems
- Risk of evidence contamination
- Reporting limitations

Interpreting and reporting malware

ROYAL HOLLOWAY UNIVERSITY OF LONDON

- Behaviour ≠ intent
- Malware presence ≠ user responsibility
- Corroboration is essential

- Describe observed behavior clearly
- Avoid speculative attribution
- State limitations explicitly
- Link findings to other artefacts

Summary

ROYAL HOLLOWAY UNIVERSITY OF LONDON

Summary

1. Malware analysis supports forensic reconstruction
2. Behaviour matters more than labels
3. Evidence is often incomplete
4. Interpretation must be cautious and contextual

Questions?

ROYAL HOLLOWAY UNIVERSITY OF LONDON

References

- W. Blunden:
  Anti-Forensics: The Rootkit Connection, Black Hat USA, 2009
- M. Hale Ligh, A. Case, J. Levy, A. Walters:
  The Art of Memory Forensics: Detecting Malware and Threats in Windows, Linux, and Mac Memory, Wiley, 2014
- W. Blunden:
  The Rootkit Arsenal: Escape and Evasion in the Dark Corners of the System, 2nd ed., Jones & Bartlett Publishers, 2012
- E. Eilam:
  Reversing: Secrets of Reverse Engineering, John Wiley & Sons, 2005
- T. Soulami:
  Inside Windows Debugging: A Practical Guide to Debugging and Tracing Strategies in Windows, Microsoft Press, 2012
- C. Anley, J. Heasman, F. Lindner, G. Richarte:
  The Shellcoder's Handbook: Discovering and Exploiting Security Holes, 2nd ed., John Wiley & Sons, 2007
- D. Vostokov:
  Memory Dump Analysis Anthology, Opentask, 2008-2011
- M. Sikorski, A. Honig:
  Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software, No Starch Press, 2012

Appendix

ROYAL HOLLOWAY UNIVERSITY OF LONDON

Binary Root Kits (1/2)

ROYAL HOLLOWAY UNIVERSITY OF LONDON

Binary or "User Land" root kits were historically the first and least sophisticated form of root kits

- Substitution or modification of binary executables to avoid detection by tools present on the target system
- Requires knowledge of all tools which may be used by an investigator, and views presented by such tools must be consistent
- Investigators may use additional tools or utilize access to "raw" information without filtering through in-system tools
- Size of the rootkit becomes an issue as tool sets become more comprehensive
- Integrity checks must also be evaded (difficult for off-line checks)

Binary Root Kits (2/2)

ROYAL HOLLOWAY UNIVERSITY OF LONDON

In many malware components, some of these techniques are (still) used

- Remote code injection (e.g., **DLL injection** on Windows systems) provides ways to control the behavior of other user-land processes
- Overwriting (*hooking*) call tables or using *detour* patching can inject code into other user processes

If known good behavior or file signatures are known, this can be identified both online and forensically

- This type of patching is, unlike binary modification and substitution purely a **live** technique!
- Root kits will target detection subsystems deliberately

Kernel Root Kits

ROYAL HOLLOWAY UNIVERSITY OF LONDON

Modification of kernel components (through various techniques from direct hooking or **Direct Kernel Object Manipulation** (DKOM) to loadable kernel modules) provides a more promising approach to establish root kits

- Hooking can occur at multiple locations, not just obvious system call and driver interfaces – these are too easy to detect in integrity checks
- Analysis of kernel data structures for anomalies or static signatures is a relatively slow process but assumes knowledge of the live behavior of the kernel

Persistent Root Kits

Persistent root kits need to survive a re-boot and hence must be re-initialized from a non-volatile origin. Example strategies (on Microsoft Windows-based systems) for this include

- Registry keys (e.g., *run keys*, file extensions)
- Startup files
- Non-existing *SafeDllSearchMode*
- Add-ons to existing applications (extension modules for applications such as web browsers)
- On-disk binary patching including of boot loader, kernel, and device drivers
- Modification of boot records
- Firmware modification